Inventory Manager: Developer Guide

Architecture, integration, deployment, and maintenance (version 1.0.0)

1. Introduction

This guide explains how the **Inventory Manager** is built, how it is started, how it is deployed, and how future developers can extend and maintain it. It is written for three readers: a developer who has just joined the project, an operations engineer who must install and run it, and a maintainer who has to decide what to change next. Each section ends with something you can run or copy, so you can check the explanation against the real system.

Inventory Manager is a Java 17 command-line application. A user can add, list, find, search, update, and delete inventory items, ask for a low-stock report, and see the total value of the stock. Everything is saved to a CSV file, so the data is still there after the program is closed and started again. The application has no third-party runtime libraries: the packaged JAR runs on any machine that has Java 17 or newer.

1.1 How the project grew

The application was built in stages, and each stage left something behind that the next one relies on.

Stage	Focus	What it added
Foundation	CRUD logic	Item model, inventory operations (create, read, update, delete), validation rules
Testing and debugging	Reliability	JUnit 5 test suite, five documented bugs reproduced and fixed
Refactoring	Structure and speed	Packages, Repository pattern, immutable Item, shared Validators, O(1) total value, cached search key
Integration and deployment	A real application	Menu interface, CSV persistence, configuration, logging, executable JAR, launcher scripts, deployment simulation, this guide

This guide documents the final stage, but it refers back to the earlier ones where a design decision only makes sense with that history.

2. Technology Stack

The stack is deliberately small. Every extra dependency is something an operations team must patch and a future developer must learn, so the project uses the JDK wherever the JDK is good enough.

Technology	Version	Used for	Why this choice
Java	17 or newer	All application code	Long-term-support release; streams with Stream.toList(), switch expressions, modern String methods

Technology	Version	Used for	Why this choice
Maven	3.8+	Build, test, JAR creation	Standard tool; one pom.xml describes the whole build
JUnit Jupiter	5.10.2	Unit and integration tests	Standard Java test framework; used only at test time
JaCoCo	0.8.12	Code-coverage report	Runs inside mvn test; report is written to target/site/jacoco
java.util.logging	JDK built-in	Application log file	No dependency to ship or patch
CSV text file	UTF-8	Data storage	Human-readable, easy to back up, no database server needed
Bash and Batch scripts	n/a	Build, package, launch, deploy checks	Works on the machines operations already has
GitHub Actions	n/a	Automatic test and package on every push	Catches breakage before it reaches a release
Docker (optional)	n/a	Container image	For teams that deploy containers

3. Architecture

3.1 The layers

The code is arranged in layers. A layer may use the layers below it and never the ones above it. That single rule is what keeps the application easy to change: you can replace the user interface or the storage without touching the business rules.

```
+-------------------------------------------------------------+
|  app.Main        composition root, command line, exit codes |
+-----------------------------+-------------------------------+
|  cli.ConsoleMenu            |  config.AppConfig             |
|  (user interface)           |  util.LoggingSetup            |
+-----------------------------+-------------------------------+
|  service.InventoryService    business rules, audit logging  |
+-------------------------------------------------------------+
|  repository.ItemRepository   (interface)                    |
|    InMemoryItemRepository  <--  CsvItemRepository (CSV file)|
+-------------------------------------------------------------+
|  model.Item (immutable)   util.Validators   exception.*     |
+-------------------------------------------------------------+
```

The actual import relationships in the source are: **app** uses cli, config, exception, repository, service, util; **cli** uses exception, model, service; **service** uses exception, model, repository; **repository** uses exception and model; **model** uses util; **config** uses exception; **util** uses nothing. There are no cycles. One was caught and removed during integration: the CSV codec was first placed in util, which made model and util depend on each other, so it was moved into repository, where it belongs.

The rule is enforced by a script rather than by good intentions. `scripts/check-layers.sh` lists which packages each package may import and fails if any import points the wrong way. It passes on the delivered source. To confirm it really detects problems, a scratch copy was edited so that InventoryService imported ConsoleMenu; the script reported `VIOLATION ... imports package`

`'cli' (not allowed from 'service')` and exited with status 1. It runs in the CI workflow before the tests.

3.2 Package responsibilities

Package	Main classes	Responsibility
app	Main	Parses command-line flags, loads configuration, builds and connects all objects, returns the exit code
cli	ConsoleMenu	Shows the menu, reads input, prints results, turns exceptions into one-line messages
config	AppConfig	Reads settings from defaults, file, environment, system property and flag, in a fixed priority order
service	InventoryService	Business rules: duplicate check, search, low-stock filter, update helper; writes the audit log
repository	ItemRepository, InMemoryItemRepository, CsvItemRepository, CsvCodec	Storage contract and its two implementations; the CSV file format
model	Item	One immutable product with validation built in
util	Validators, LoggingSetup	Shared validation rules; log file and console configuration
exception	ItemNotFoundException, DuplicateItemException, RepositoryException	Named failures that upper layers can handle

3.3 Key design decisions

- **Repository pattern.** InventoryService depends on the ItemRepository interface, not on a file or a database. The CSV store was added in this stage without changing a single business rule, which is the proof that the boundary is in the right place.
- **Immutable Item.** An Item cannot be changed after creation. To change a quantity you call `item.withQuantity(7)`, which returns a new, validated Item. This guarantees that an invalid item can never exist, and it makes the repository's running total safe, because a stored item cannot change value behind the repository's back.
- **Composition root.** Only Main knows which concrete classes are used. Every other class receives its collaborators through its constructor. To switch to a database you change one line in Main.
- **Exceptions as a contract.** Validation problems throw IllegalArgumentException; a missing id throws ItemNotFoundException; a duplicate throws DuplicateItemException; storage problems throw RepositoryException. The menu catches exactly these and nothing broader, so real bugs (for example a NullPointerException) are never hidden behind a friendly message.
- **Write first, then apply.** The CSV repository writes the new complete state to disk before it changes memory. If the write fails, memory is untouched. This removes the whole class of "memory and file disagree" bugs without needing any rollback code.

3.4 Worked example: adding an item

Following one request through the layers shows how the pieces cooperate. The user chooses option 1 and enters `A2`, `Cable, USB-C`, `3`, `199.5`.

1. **ConsoleMenu.addItem()** reads four lines, converts the numbers (a non-number becomes a friendly IllegalArgumentException), and calls `new Item(...)`.
2. **Item's constructor** asks Validators to check each field. Blank text, line breaks, negative quantity, and NaN or infinite prices are rejected here, once, for every caller.
3. **InventoryService.addItem()** asks the repository whether the id already exists. If it does, it throws DuplicateItemException.
4. **CsvItemRepository.save()** builds the new full item list, writes it to `inventory.csv.tmp`, and atomically renames it over `inventory.csv`.
5. **Only then** does the in-memory repository update its map and its running total of stock value.
6. **InventoryService** writes `Added item A2` to the log, and ConsoleMenu prints `Added: Item{...}`.

The resulting line in the data file shows the CSV quoting rule: a field containing a comma is wrapped in double quotes.

```
id,name,quantity,price
A1,Laptop,10,50000.0
A2,"Cable, USB-C",3,199.5
```

4. Application Workflow

4.1 Startup sequence

1. `Main.run` reads the flags (`--config=`, `--data=`, `--version`, `--help`). An unknown flag stops the program with exit code 2.
2. `AppConfig.load` resolves the settings (section 5). A missing config file or an invalid threshold stops the program with exit code 1 and the message `Startup failed: ...`.
3. `LoggingSetup.init` opens the log file. If the log file cannot be opened the application still starts and logs to the console only.
4. `CsvItemRepository` reads the data file. A missing file means a fresh install and an empty inventory. A corrupt file stops startup with the file name and line number.
5. The service and menu are created and the menu loop begins.

4.2 A real session

The following transcript was produced by running the packaged application with scripted input. It adds two items, tries a quantity that is not a number, tries a negative quantity, searches, shows the low-stock report, shows the total, and exits. Two things differ from what you see on a terminal: the menu, which is printed before every prompt, is left out to keep the transcript readable, and the typed answers are not echoed because the input was piped in. The install path on the first line was also shortened.

```
Data file: /srv/inventory-app-1.0.0/data/inventory.csv
Loaded 0 item(s).
=== Inventory Manager ===
```

```
Choose an option: Item id: Name: Quantity: Price: Added: Item{id='A1', name='Laptop',
qty=10, price=50000.00}
Choose an option: Item id: Name: Quantity: Price: Added: Item{id='A2', name='Cable,
USB-C', qty=3, price=199.50}
Choose an option: Item id: Name: Quantity: Error: 'abc' is not a valid whole number
Choose an option: Item id: New quantity: Error: Quantity must not be negative
Choose an option: Search text: ID         NAME                       QTY
PRICE          VALUE
A1         Laptop                      10     50000.00      500000.00
Choose an option: Threshold [5]: Items with quantity below 5:
ID         NAME                       QTY        PRICE          VALUE
A2         Cable, USB-C                 3       199.50         598.50
Choose an option: Total inventory value: 500598.50
Choose an option: Goodbye.
```

Notice that the two errors did not stop the program and did not change any data. Each error is one line starting with `Error:`.

4.3 Exit codes

Code	Meaning	Typical cause
0	Normal exit	User chose 0, or input ended (Ctrl+D, or the end of a piped script)
1	Startup failure	Config file not found, invalid setting, corrupt or unreadable data file
2	Bad command line	Unknown flag such as `--bogus`

Exit codes matter to operations: a service manager or a script can tell "the program ran and finished" from "the program refused to start" without reading any text.

5. Configuration

5.1 Settings

Key	Default	Meaning
data.file	data/inventory.csv	Where the inventory is stored
log.file	logs/inventory.log	Application log; INFO and above are written
low.stock.threshold	5	Items with quantity strictly below this number appear in the low-stock report
app.name, app.version	Inventory Manager, 1.0.0	Shown by `--version`

5.2 Priority order

The data file location can come from five places. When more than one is present, the later one in this list wins.

1. Defaults bundled inside the JAR
2. External file given with `--config=PATH`
3. Environment variable `INVENTORY_DATA_FILE`

4. JVM option `-Dinventory.data.file=PATH`
5. Command-line flag `--data=PATH` (highest priority)

```
# Use the bundled defaults
java -jar lib/inventory-app-1.0.0.jar

# Point at a different data file for one run
bin/run.sh --data=/srv/inventory/stock.csv

# Set it for a whole server session
export INVENTORY_DATA_FILE=/srv/inventory/stock.csv
bin/run.sh
```

> Windows paths in a .properties file must use forward slashes (C:/inventory/data/inventory.csv). A single backslash is an escape character in that file format and would silently corrupt the path. Paths given with `--data=` on the command line are not affected.

The precedence rules are covered by automated tests in AppConfigTest. They pass the environment and system properties in as parameters, so the tests never depend on the real machine's environment.

6. Data Storage

6.1 The CSV format

The file has a header line and one line per item with four fields: id, name, quantity, price. A field that contains a comma or a double quote is wrapped in double quotes, and a quote inside a quoted field is written twice. These are the standard CSV rules, so the file opens correctly in Excel or any text editor.

```
id,name,quantity,price
A1,Laptop,10,50000.0
A2,"Cable, USB-C",25,199.5
A3,"15"" Monitor",4,8999.0
```

Prices are written with Java's locale-independent number format, so a file written on a computer configured for comma decimals still reads correctly on any other computer. Text is UTF-8, so names in any language are safe, and the repository tests include a Hindi name to prove it.

6.2 Safety guarantees

- **Atomic replacement.** Each save writes `inventory.csv.tmp` and then renames it over `inventory.csv`. A crash or power cut leaves either the old complete file or the new complete file, never half of one.
- **Consistency on failure.** The disk write happens before the memory change. A test replaces the disk with one that always fails and checks that adding, updating, and deleting all leave the in-memory data exactly as it was.
- **Loud corruption handling.** A bad line stops startup with a precise message instead of silently dropping an item. The application never "repairs" data on its own.

```
Startup failed: Corrupt data file data/inventory.csv at line 3: Invalid number (For
input string: "many")
```

The loader accepts a file without a header, skips blank lines, and handles Windows (CRLF) line endings. It rejects duplicate ids, because two lines with the same id would otherwise let the later one silently replace the earlier one.

7. Integration Notes

Putting independently developed modules together exposed interactions that were invisible while each module was tested alone. These were identified by reviewing every boundary before and during the integration work. Each one has a resolution and an automated test, so a future change that reintroduces it will be caught.

Integration issue	Why it matters	Resolution	Covered by
Loading from file could trigger a write per row, or bypass the running total	Slow start-up, or a wrong stock value after a restart	The loader calls the parent class's save directly, so no disk write happens and the total is maintained	CsvItemRepositoryTest: survivesRestart (total 501500.00)
Disk failure after a memory change	Memory and file disagree; data is lost on next restart	Write first, then apply. No rollback needed	Three failing-disk tests (add, update, delete)
A line break inside a name would split one item into two CSV lines	Corrupt file on next start	Validators rejects line breaks at the model level, so every storage back-end is protected	ValidatorsTest, ItemTest, CsvCodecTest
File I/O throws checked IOException	Storage details leak into the service and UI	Translated into the unchecked RepositoryException at the repository boundary	CsvItemRepositoryTest: directoryAsDataFile
Relative paths depend on the folder the user launched from	Data written to a different place each run	Launcher scripts switch to the install folder before starting Java	deploy-simulate.sh (restart check)
Number formatting depends on the machine language	A price saved as "1,5" could be misread	Locale-independent formatting in the CSV and the display	CsvCodecTest: extremePrices
User typos could crash the menu	Interactive tool unusable	Menu catches the known exception types and continues	ConsoleMenuTest (error cases)
Windows backslashes in .properties	Path silently altered	Documented; integration tests write config with Properties.store, which escapes correctly	MainIntegrationTest

8. Build, Test, and Package

8.1 Commands

```
bash scripts/check-layers.sh    # verify the package dependency rules
mvn test                      # compile, run all tests, write coverage report
bash scripts/build.sh         # build target/inventory-app-1.0.0.jar
bash scripts/package.sh       # build + assemble dist/inventory-app-1.0.0.zip
bash scripts/deploy-simulate.sh   # acceptance checks on the zip
```

build.sh uses Maven when it is installed. If it is not, or if you pass `--no-maven`, it falls back to `javac` and `jar` so a machine with only a JDK can still build from source. The version number is read from pom.xml, so there is one place to change it. **package.sh** builds the JAR, assembles the folder layout below, zips it, and writes a SHA-256 checksum file next to the zip.

8.2 The release bundle

```
inventory-app-1.0.0/
  README.txt                     operations quick start
  bin/run.sh, bin/run.bat        launchers
  conf/application.properties    editable settings
  lib/inventory-app-1.0.0.jar    the application (about 27 KB)
  data/                          inventory.csv appears here at runtime
  logs/                          inventory.log appears here at runtime
```

The JAR's manifest names `com.inventory.app.Main` as the main class, so `java -jar` works with no extra arguments. Because the application has no runtime dependencies, the JAR is self-contained.

9. Deployment Guide

9.1 Linux or macOS

1. Check Java: `java -version` must report 17 or newer.
2. Copy `inventory-app-1.0.0.zip` and its `.sha256` file to the server.
3. Verify the download: `sha256sum -c inventory-app-1.0.0.zip.sha256` must print `OK`.
4. Unpack: `unzip inventory-app-1.0.0.zip -d /opt`.
5. Start: `/opt/inventory-app-1.0.0/bin/run.sh`.

The launcher finds its own install folder, checks the Java version and prints a clear error if Java is missing or too old, switches into the install folder, and starts the JAR with the bundled config. Any extra flags are passed through, and `JAVA_OPTS` (for example `-Xmx256m`) is honoured.

9.2 Windows

Unzip the bundle, open Command Prompt in the folder, and run `bin\run.bat`. It performs the same steps as the Linux launcher.

> Verification status: run.bat was written to mirror run.sh but has not been executed, because the authoring environment is Linux. Run `bin\run.bat --version` as the first acceptance check on Windows.

9.3 Container

The Dockerfile copies the unpacked bundle into a Java 17 runtime image, runs as a non-root user, and declares the data folder as a volume so data survives when the container is replaced.

```
bash scripts/package.sh
docker build -t inventory-app:1.0.0 .
docker run -it --rm -v inventory-data:/opt/inventory/data inventory-app:1.0.0
```

> Verification status: the Dockerfile has not been built, because no Docker daemon was available where this project was authored. Treat the first image build as part of acceptance.

9.4 Simulated hand-over to operations

`scripts/deploy-simulate.sh` plays the role of an operations engineer receiving the zip. It verifies the checksum, unpacks into a clean temporary folder, runs smoke tests, uses the application, restarts it to prove the data survived, and confirms that bad input is refused with the right exit code. This is its real output from the final build:

```
[1/7] Verify checksum
inventory-app-1.0.0.zip: OK
[2/7] Unpack on the 'server'
  PASS  launcher is executable
  PASS  JAR present in lib/
  PASS  config present in conf/
[3/7] Smoke test: --version and --help
  PASS  version banner is printed
  PASS  --help exits 0
[4/7] Session 1: add two items (first run, no data file yet)
  PASS  session 1 exit code is 0
  PASS  session 1 reported 0 items loaded
  PASS  data file was created
  PASS  data file has header + 2 rows
  PASS  comma in name was quoted in CSV
[5/7] Session 2: RESTART - data must still be there
  PASS  session 2 exit code is 0
  PASS  session 2 loaded 2 items
  PASS  total value is 500598.50
  PASS  log file records the additions
[6/7] Corrupt data file must be refused with a clear message
  PASS  corrupt file gives exit code 1
  PASS  message names the line number
[7/7] Bad command line must be refused
  PASS  unknown flag gives exit code 2

Deployment simulation: 17 passed, 0 failed
```

The same script runs in the GitHub Actions workflow after every push, so a release bundle that fails these checks is never published.

9.5 Backup, upgrade, and rollback

- **Backup.** All data is one file. Stop the application and copy `data/inventory.csv`. Keep dated copies, for example `inventory-2026-10-26.csv`.
- **Upgrade.** Stop the application, back up the data file, unzip the new version into a new folder beside the old one, copy `data/` and `conf/` across, start the new version, and run `--version` plus a quick list as a check.
- **Rollback.** Stop the application and start the previous version's folder. Its own data folder was never touched, so rollback takes seconds. If the new version had already written to a shared data file, restore the backup taken before the upgrade.

10. Testing Strategy

10.1 The suite

Test class	Tests	What it proves
ItemTest	12	Validation, immutability, equality by id, cached search key
ValidatorsTest	6	Each shared rule, including the line-break rule
CsvCodecTest	11	CSV round trips, quoting, extreme prices, malformed lines
InMemoryItemRepositoryTest	8	Storage contract and running total
CsvItemRepositoryTest	17	Persistence, restart, order, UTF-8, corruption, failing disk
InventoryServiceTest	27	All CRUD rules, search, low stock, totals, dependency injection
AppConfigTest	8	Defaults, file, environment, system property, flag priority; bad values
ConsoleMenuTest	24	Every menu command and every error path, driven by scripted input
MainIntegrationTest	9	Whole application: restarts, error handling, exit codes, log file
Total	122	

The tests are in three layers. Unit tests check one class in isolation. Component tests such as CsvItemRepositoryTest use the real file system in a temporary folder. The integration tests in MainIntegrationTest call `Main.run` exactly as the operating system would, simulate a restart by calling it again, and check the files it leaves behind. The menu is easy to test this way because input and output streams are passed in rather than hard-wired to the keyboard and screen.

10.2 An example worth copying

This test simulates a full disk by overriding the one method that writes, then checks that the failed operation leaves nothing half-done:

```
private static CsvItemRepository failingDisk(Path file) {
    return new CsvItemRepository(file) {
        @Override
        protected void write(List<Item> items) {
            throw new RepositoryException("disk full");
        }
    };
}

@Test
void failedUpdateKeepsOldItem() throws Exception {
    Path file = newDataFile();
    new CsvItemRepository(file).save(new Item("A1", "Laptop", 10, 100));

    CsvItemRepository repository = failingDisk(file);
    assertThrows(RepositoryException.class,
            () -> repository.save(new Item("A1", "Laptop", 99, 100)));
    assertEquals(10, repository.findById("A1").orElseThrow().quantity());
    assertEquals(1000.0, repository.totalValue(), 0.0001);
}
```

10.3 Do the tests really catch bugs?

A test suite that has never failed proves little, so the failure-consistency tests were checked with a deliberate fault. In a scratch copy, the order inside `CsvItemRepository.save` was reversed so that memory changed before the disk write. The unmodified code passed all 17 repository tests; the faulty copy failed exactly the two tests that cover this rule (add and update). The fault was never in the delivered code.

10.4 Results and how to reproduce them

All 122 tests passed on the final code, and all 17 deployment checks passed. One honest limitation: the authoring environment could not download JUnit or JaCoCo from Maven Central, so the tests were executed with a small stand-in runner that implements the JUnit annotations and assertions the tests use. They use no other JUnit features. Run `mvn test` on your own machine for the official JUnit result and the JaCoCo coverage report (`target/site/jacoco/index.html`); line-coverage percentages were not measured here and are deliberately not quoted.

11. Extending the Application

The examples below show the intended way to grow the system. They are illustrative sketches written for this guide; they are not part of the delivered source and were not compiled.

11.1 Add a field (for example, category)

1. **model.Item:** add a `category` field, validate it with `Validators.requireNonBlank`, and add `withCategory(...)`. Because Item is immutable, no other class can create an invalid category.
2. **repository.CsvCodec:** write a fifth column. For backward compatibility, make `decode` accept the old four-field lines and use a default such as `General`, so existing data files keep loading.
3. **cli.ConsoleMenu:** add one prompt in `addItem()` and one column in `printTable()`.
4. **Tests:** extend ItemTest for the new rule, CsvCodecTest for old and new line formats, and one MainIntegrationTest that restarts and checks the category survived.

```
// CsvCodec.decode - accept both file generations
if (fields.size() != 4 && fields.size() != 5) {
    throw new IllegalArgumentException(
        "Expected 4 or 5 fields but found " + fields.size());
}
String category = fields.size() == 5 ? fields.get(4) : "General";
```

11.2 Add a new storage back-end (database)

Implement the ItemRepository interface and change one line in Main. Nothing in service, cli, or model changes.

```
public class JdbcItemRepository implements ItemRepository {
    private final DataSource dataSource;

    public JdbcItemRepository(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override public Optional<Item> findById(String id) {
```

```
        // SELECT id, name, quantity, price FROM items WHERE id = ?
    }
    @Override public double totalValue() {
        // SELECT COALESCE(SUM(quantity * price), 0) FROM items
        // the database computes this, so no running total is needed
    }
    // ...the remaining interface methods follow the same pattern
}

// Main.java - the only line that changes:
ItemRepository repository = new JdbcItemRepository(dataSource);
```

Reuse the contract tests: copy InMemoryItemRepositoryTest, point it at the new class, and every behaviour the service relies on (replace on same id, delete returns true once then false, order of results) is checked automatically.

11.3 Add a menu command

Adding "export low-stock report to a file" touches two places. Put the logic in the service, because it is a business operation, and only the prompt in the menu.

```
// InventoryService
public List<String> lowStockReport(int threshold) {
    return getLowStockItems(threshold).stream()
            .map(i -> i.id() + "," + i.name() + "," + i.quantity())
            .toList();
}

// ConsoleMenu.dispatch - add a case and a private method
case "11" -> exportLowStock();
```

11.4 Add a configuration key

1. Add a default to `src/main/resources/application.properties` and to `deploy/conf/application.properties`.
2. Read and validate it in `AppConfig`, and expose it through an accessor.
3. Pass it to the class that needs it from `Main`. Do not let that class read the properties itself.
4. Add a case to `AppConfigTest` for the default, an override, and an invalid value.

12. Maintenance and Scalability

12.1 Known limitations

Every design has limits. These are the ones a maintainer should know before they cause a surprise in production.

Limitation	Impact	Mitigation or next step
Every change rewrites the whole CSV file (O(n))	Measured cost of one save: about 4 ms at 1,000 items, 17 ms at 10,000, 49 ms at 100,000, 173 ms at 500,000 (cloud sandbox, JDK 21; your disk will differ). Fine for one person at a keyboard, but importing 10,000 items one by one would take well over a minute	Add a batch-save method for imports; for large or shared data move to a database back-end (section 11.2) or an append-only log with periodic compaction
No locking between processes	Two copies of the program on one file: the last writer wins and earlier changes are lost	Run one instance per data file; add a lock file or use a database before sharing
Not thread-safe	Safe for the single-threaded menu; unsafe if reused behind a web server	Synchronise the repository or give each request its own transaction before adding concurrency
Prices are `double`	Floating-point rounding can produce tiny drift in totals (for example 0.1 + 0.2)	Use BigDecimal or integer minor units (paise or cents) before real accounting use
Interactive console only	No remote or automated access	Add a REST layer above InventoryService; the service already has no UI dependency
Log file grows without limit	Disk fills over a long time	Use FileHandler rotation parameters, or an external rotation tool

12.2 Suggested roadmap

1. **Next (small):** log rotation; a `--check` flag that validates the data file and exits; BigDecimal prices.
2. **Then (medium):** a database repository using the contract tests from section 11.2; an import/export command for CSV files from other systems.
3. **Later (large):** a REST API and role-based access; concurrent users; containerised deployment with a managed database.

12.3 Maintenance practices

- **Keep the layer rule.** Run `bash scripts/check-layers.sh` before every pull request. When you add a package, add it to the table at the top of that script in the same change.
- **Tests first for bugs.** When a bug is reported, write the failing test before fixing it, as in the earlier debugging stage. The test then guards against a repeat.
- **Version with intent.** Use semantic versioning (1.0.0, 1.1.0 for a compatible feature, 2.0.0 for a break such as a data-file format change). The version lives in pom.xml and is read by the scripts.
- **Treat the data-file format as an API.** Any change must still load old files, as shown in section 11.1, and the change must be listed in release notes.
- **Keep dependencies at zero where possible.** The application ships none. Test-only dependencies (JUnit, JaCoCo) should be updated at least every few months.

- **Let CI be the gate.** A change is not done until `mvn test`, packaging, and the deployment simulation all pass in GitHub Actions.

13. Troubleshooting

Symptom	Likely cause	What to do
`ERROR: Java was not found on PATH`	Java is not installed or not on PATH	Install a JRE or JDK 17+, open a new terminal, run `java -version`
`ERROR: Java 17 or newer is required (found: 1)`	Java 8 is the default `java`	Point PATH or JAVA_HOME at Java 17+
`Startup failed: Config file not found: ...`	Wrong `--config=` path	Use an absolute path, or omit the flag to use bundled defaults
`Startup failed: Corrupt data file ... at line N`	Hand-edited or truncated CSV	Open the file, fix line N (or restore a backup), restart
`Startup failed: low.stock.threshold must be a whole number`	Invalid value in the properties file	Set a whole number, 0 or greater
Changes appear to vanish after restart	Application started from a different folder, or `--data=` points elsewhere	Use the launcher scripts; read the `Data file:` line printed at startup
`Error: Item already exists: A1`	Id is already used	Pick another id, or update the existing item
No log file appears	Log folder not writable	A warning is printed to the console; fix permissions on `logs/`
Path with backslashes behaves oddly	Backslash is an escape in .properties	Use forward slashes in properties files

14. Quick Reference

Menu option	Action
1	Add item (id, name, quantity, price)
2	List all items
3	Find an item by id
4	Search by name (case-insensitive, partial match)
5, 6, 7	Update quantity, update price, rename
8	Delete item
9	Low-stock report (blank answer uses the configured threshold)
10	Total inventory value
0	Exit

Command-line flag	Effect
`--config=PATH`	Load an external properties file
`--data=PATH`	Use this CSV file (highest priority)
`--version`, `-v`	Print name and version, exit 0
`--help`, `-h`	Print usage, exit 0

This guide, the source code, the tests, and the scripts together form the hand-over package. If the guide and the code ever disagree, the code and its tests are correct and the guide should be updated in the same change.